

FIT3143 Lab #1 Assessment (Week 4)

THREADS & OPENMP

OBJECTIVES

- The purpose of this lab assessment is to introduce you to POSIX Threads and OpenMP
- Design and develop parallel algorithms for shared memory parallel computing

MARKS

- The lab session is worth 8 of the unit's final mark.

LAB INSTRUCTIONS

1. Preparation is required for this Lab session. Please **DO NOT** plan to complete the Lab without any preparation/understanding.
2. Students should start working on the Lab tasks at least 1 week prior to the assessed session.
3. Students must submit their answers and codes via Moodle in the required format before the start of the allocated class. A **late penalty** (5% per day) will apply if you do not submit the required materials on time!
4. In addition to regular offline/off-class marking, students' (or Team's) slides, answers, and/or source code will also be reviewed and assessed through in-class presentations, peer reviews, and/or question-and-answer sessions.
5. For **team/group presentations**, marks in general will be allocated for:
 - The correctness of the codes or results
 - The quality and clarity of the presentation
 - The correctness of the explanation during the presentation

The presentation content should include slides that are clear, easy to understand, and readable in a professional working environment (aka an industry setting). While the teaching team does not impose strict limits on the presentation format (slides/document format), nevertheless, if the presentation is hard to read/digest (e.g., speech is misaligned with the presented content, or slides/speech are unclear, or arguments are

made without sound logic), marks will be deducted. Both the quality and correctness of your submitted/demonstrated work are assessed and graded.

6. For **individual/group interviews and Q&A sessions**, marks will be allocated for:

- The correctness of the verbal answer
- The quality of the verbal answer

Focus on the most important idea in the answer. Marks will be awarded on the student's ability to explain their ideas with reference to the submitted code or documents in a concise, focused and accurate manner. Answers with errors, inaccurate or lengthy explanations will lose marks. Showing little to no understanding of the code will result in a loss of marks. Showing little to no understanding of the associated libraries, or submitting results will result in a loss of marks. Showing little to no understanding of how to select and filter information (aka dumping unfiltered answers without processing from the textbook/internet/AI agent) will lose marks.

- 7. Always make sure you read and follow the marking rubric well in advance of the deadline.
- 8. Marks will not be awarded if you skip the class, or do not make any submissions, or do not make any contributions in the working group/team, or make an empty submission to Moodle (unless a special consideration extension has been approved).
- 9. You are allowed to use Generative-AI to search for information and resources during the preparation period. However, you must declare in your report and upload all the prompt records (in PDF files).
- 10. AI tools are not allowed during the presentation period or any oral/coding interview sessions.
- 11. All submitted files should ideally include students' names, ID and Monash email addresses.

LAB IN-CLASS ASSESSMENT ACTIVITIES

Students are required to form teams of 2.

Note: For students who cannot form teams due to exceptional circumstances, arrangements must be made with the teaching team (generally the lead staff member in the corresponding applied/lab class) on/before Week 3.

- ❖ Team Preparation Period (before deadline): Divide the workload among team members and complete all the tasks. For each team, you must submit the required files (e.g., report, source code, slides, and corresponding documents) before the deadline.
- ❖ Team Presentation Period (5 min): For each team, you are required to present and demonstrate your code, results and observations during the lab session. All team members must be present in the lab session.
- ❖ Q&A Period (2 min): Each team member will be asked one to two questions based on the submitted and/or presented work.

Note: If the required submissions are missing or any member(s) of the student team are absent, no marks will be given to the assessment.

Task 1 – Serial Code - Finding Prime Numbers

Write a serial C program to search for prime numbers that are strictly less than an integer n , provided by the user. The program will output a sorted list of all prime numbers found.

Example: For instance, if the user inputs n as 10 on the terminal, the prime numbers being printed are: 2, 3, 5, 7 (sorted in ascending order).

Your program is required to have the capability to print the sorted list of prime numbers to:

- a) the standard output (for small n values, e.g., $n < 100$), and
- b) a text file (for larger n values, e.g., $n > 100$).

Hint:

- How to check if a number, let's say k , is prime? Yes, you can check if k is divisible by 2, by 3, by 4, by 5, ..., by $k-1$.
- However, there is a (slightly) smarter way. Imagine k is not a prime number, there must exist two integers m and n such that m times n equals k . Will both m and n be larger than the square root of k ? No! If m is larger than the square root of k , then n must be smaller than the square root of k , right?
- In other words, you don't need to check until $k - 1$ (think what happens when k is really large).
- For C language: Make sure you know how to use the `sqrt()` of `math.h` and compile using the `-lm` flag of `gcc`.

Implement the program ("task1.c") that can search for prime numbers with n at least up to 10,000,000. Your program should also be capable of measuring and printing the time being used to execute the program.

Reminder: Your serial code must be clean and bug-free so you can reliably compare your parallel version (Tasks 2 and 3) against the serial version (Task 1). The code should also be well-formatted so that you can easily explain your approach and/or demonstrate your code in Task 4.

Task 2 – POSIX Threads - Finding Prime Numbers

Write and implement a parallel version ("task2.c") of your serial code in C utilising POSIX Threads.

In this part, your team will need to design a parallel partitioning scheme to distribute the workload among the threads and implement it in C.

After you implement the task, measure again the time required to search for prime numbers less than an integer n and compute the speed-up by your parallel implementation. Make sure your code can still produce a sorted list of prime numbers (in ascending order). You should test for different values of n and tabulate the serial and parallel computation times. Based on the tabulated results, you can compute the speedup for different n values. We recommend that you first test with $n > 10,000,000$ and then increase n .

Common questions you need to consider:

- What is the speed-up? Is it reasonable? If not, is the run time measurement correct?

- How would the speed-up change when you increase & decrease the number of threads and the size of n ? Why?
- How do you distribute the tasks to the threads to ensure balanced workload distribution? Is it a good approach?

Reminder: Your code must be clean and bug-free so that you can *reliably compare your parallel version against the serial version (Task 1)*. Your code should also be well-formatted so that you can easily explain your approach and/or demonstrate your code in Task 4.

Task 3 – OpenMP - Finding Prime Numbers

Write and implement a parallel version ("task3.c") of your serial code in C utilising OpenMP.

In this part, your team will need to design a parallel partitioning scheme to distribute the workload using OpenMP and implement it in C.

After you implement the task, measure again the time required to search for prime numbers less than an integer n and compute the speed-up by your parallel implementation using the same method in Task 2. Make sure your code can still produce a sorted list of prime numbers (in ascending order).

Common questions you need to consider:

- What is the speed-up? Is it reasonable? If not, is the run time measurement correct?
- How would the speed-up change when you increase & decrease the number of threads and the size of n in OpenMP? Why?
- How do you distribute the tasks to the threads to ensure balanced workload distribution? Is it a good approach?

Reminder: Your code must be clean and bug-free so that you can *reliably compare your parallel version against the serial version (Task 1)*. Your code should also be well-formatted so that you can easily explain your approach and/or demonstrate your code in Task 4.

Task 4 – Presentation slides and documentations

In this task, you are required to prepare presentation slides (or documentation) to report and present your answers and results from previous Tasks. You will have 7 minutes to present your slides and 1 to 2 minutes for the Q&A session.

Your presentation slides (or documentation) should have the following sections and content (at minimum):

- a) (Task 1) Serial code [Estimated Presentation Time: 1 min]
The section introduces your approach to implementing the serial code to find prime numbers.
- b) (Task 2) POSIX Threads [Estimated Presentation Time: 3 min]
The section introduces your approach to implementing the parallel version with POSIX Threads. You are required to discuss your parallel partitioning scheme (workload distribution) and produce graphs comparing your Task 2 implementation against Task 1.
You are required to produce the following graphs:
 - 1) Comparing the run time of serial code vs parallel code (POSIX Threads) with increasing size of n .
 - 2) Showcasing the speedup of the parallel code (POSIX Threads) with increasing size of n .

You are also required to produce the following graphs:

- 3) Comparing the run time of serial code vs parallel code (POSIX Threads) with an increasing number of threads.
- 4) Showcasing the speedup of the parallel code (POSIX Threads) with an increasing number of threads.

c) (Task 3) Open MP [ETA: 2 min]

The section introduces your approach to implementing the parallel version with OpenMP. You are required to discuss your parallel partitioning scheme (workload distribution) and produce graphs comparing your Task 3 implementation against Task 1 and Task 2.

You are required to produce the following graphs:

- 5) Comparing the run time of serial code vs parallel code (OpenMP) with increasing size of n .
- 6) Showcasing the speedup of the parallel code (OpenMP) with increasing size of n .

You are also required to produce the following graphs:

- 7) Comparing the run time of parallel code (POSIX Thread) vs parallel code (OpenMP) with increasing size of n .
- 8) Comparing the run time of parallel code (POSIX Thread) vs parallel code (OpenMP) with an increasing number of threads.

d) Conclusion and Recommendations [ETA: 1 min]

The section concludes the presentation and discusses major issues, limitations, and future work.

Note: To obtain statistically convincing evidence, you should test with at least 30 different numbers of n . You may also need to increase n further to obtain significant results (depending on your machine hardware).

Note: Make sure you test the number of threads from 1 up to at least the number of cores available in your CPU. What would happen if you increase the number of threads more than the number of cores available in your machine?

e) Q&A [ETA: 1 to 2 min]

The section invites questions from the audience (TA/makers). The teaching team will ask questions related to your presentation and the submitted files.

Note: Practice makes perfect. If you are still deciding what to present and searching for information/content from the slides, chances are you will not be able to present within the time limit.

Note: You are allowed to put extra slides as an appendix (after the Q&A section) for your own convenience. Make sure they are clearly labelled so that markers will not be confused or mix them up with the other required sections.

Submission Checklist

- Coding Tasks (Task 1 to 3) – task1.c, task2.c, and task3.c
- Task 4 - Presentation slides (or documentation)
 - The documentation can be in the format of slides, docx, PDF or an equivalent format.
 - Please aim to keep the slides or documentation to within an 8-minute presentation (including the Q&A session).
 - Focus on emphasising the design, results and observations that would demonstrate a good understanding of the lab tasks.
- A separate AI declaration (in PDF format), **if not** already declared in the submitted documentation as aforementioned.

LAB INSTRUCTIONS FOR ALTERNATIVE ASSESSMENT

This section only applies if you or your team member(s) are **granted a special consideration extension** by the special consideration team (i.e., with a valid approval notice).

1. As a reminder, note that the regular 2 day extensions would not apply for applied/lab sessions (as they are in-class assessments with group/team works).
2. Instead of reviewing student's answers (in-class) in a presentation, peer review, and/or question and answer format during the regular in-class session, **student's answers (Task 4) will be assessed based on a recorded presentation video**. The video is required to be less than 9 minutes, and markers will be marking the assessment offline.
3. Markers will only mark the first 9 minutes of the video at regular playback speed. If students decide to compress the video by speeding up the playback speed, presentation quality could be affected and marks may be deducted according to the marking rubric.
4. Instead of having an in-person Q&A sessions, students will need to answer the following two questions in their recording:
 - Is the speed-up exactly equal to the number of threads you created/used in your POSIX Thread implementation (Task 2)? Why are they the same/different? Explain at least 2 reasons.
 - Would you recommend Open MP over POSIX Thread? Why/Why not? Explain your reasons.
5. Apart from the recorded video file (mp4), students are still required to submit the presentation slides (PDF/pptx), source codes, and any AI declarations/reports (PDF) listed in the regular submission checklist via Moodle before **the approved extended due date**.
6. **A late penalty** (5% per day) will still apply if you do not submit the required materials on time (against the new extended date)! As a reminder, if you submit 7 days after the extended due date, you will still receive a 0 mark.
7. AI tools are generally not allowed during the recording sessions.
8. Before you start your presentation, you are required to show your face and your student ID in the video.